

AngularJS, HTML and CSS Learning Guide

A separate frontend guide based on the Byte Bazaar project.

1. What This Guide Covers

This guide is for understanding the frontend part of Byte Bazaar. It assumes you already know basic HTML and CSS, but you are new to AngularJS. The guide explains the AngularJS concepts used in the project and the HTML/CSS layout techniques used to build the page.

The frontend files are:

```
src/main/resources/static/index.html
src/main/resources/static/styles.css
src/main/resources/static/app.js
```

2. How Frontend Works in This Project

The frontend is a single page. It does not use separate pages for catalog, login, cart, and orders. Instead, all sections are present in `index.html`, and AngularJS shows or hides sections based on `store.view`.

```
store.view = 'catalog';
```

If `store.view` is `catalog`, the catalog section is visible. If it is `cart`, the cart section is visible.

3. AngularJS in Simple Words

AngularJS connects HTML with JavaScript data. In plain JavaScript, you normally select elements and manually change them. In AngularJS, the HTML directly reads values from JavaScript variables.

Example idea:

```
// JavaScript data
store.user = { name: 'Varun' };

// HTML output
Signed in as {{store.user.name}}
```

AngularJS automatically prints the value in the page.

4. AngularJS Application Setup

HTML Side

```
<html lang="en" ng-app="byteBazaarApp">
```

ng-app starts AngularJS for this page. The name byteBazaarApp must match the app name in JavaScript.

JavaScript Side

```
angular.module('byteBazaarApp', [])
```

This creates the AngularJS module. The empty array means this app has no extra AngularJS dependencies.

5. AngularJS Controller

HTML Side

```
<body ng-controller="StoreController as store">
```

This connects the HTML body to StoreController. The controller is used as store inside HTML.

JavaScript Side

```
.controller('StoreController', function ($http, $timeout) {  
    var vm = this;  
});
```

\$http is used for API calls. \$timeout is used for clearing messages after a few seconds. var vm = this stores the controller object in a variable.

In this project, vm in JavaScript is the same object that HTML calls store.

6. Data Binding

Data binding means connecting frontend data with HTML. AngularJS updates the page when JavaScript values change.

One-Way Display Binding

```
{{store.message}}
```

This prints the value of store.message in HTML.

Two-Way Form Binding

```
<input type="email" ng-model="store.loginForm.email">
```

ng-model connects the input box and JavaScript variable. When the user types, the JavaScript value changes. If JavaScript changes the value, the input also updates.

7. Directives Used in Byte Bazaar

Directive	Purpose	Project Example
ng-app	Starts AngularJS app.	ng-app="byteBazaarApp"
ng-controller	Connects JavaScript controller with HTML.	ng-controller="StoreController as store"
ng-click	Runs function on button click.	ng-click="store.show('cart')"
ng-model	Connects form input with JavaScript variable.	ng-model="store.search"
ng-repeat	Repeats HTML for each item in an array.	ng-repeat="product in store.products"
ng-if	Adds/removes HTML based on condition.	ng-if="store.user"
ng-show	Shows/hides HTML based on condition.	ng-show="store.view === 'catalog'"
ng-submit	Runs function when form is submitted.	ng-submit="store.login()"
ng-disabled	Disables button based on condition.	ng-disabled="!product.stock"
ng-class	Adds class dynamically.	ng-class="{active: store.view === 'catalog'}"

8. Understanding ng-repeat

ng-repeat is used when you have a list. In Byte Bazaar, products come from the backend as an array. AngularJS creates one product card for every product.

```
<article class="product-card"
  ng-repeat="product in store.products | filter:store.productFilter">
  <h2>{{product.name}}</h2>
</article>
```

Here, product is a temporary variable for one item from store.products.

9. Understanding Filters

AngularJS filters change or limit displayed data. This project uses:

```
store.products | filter:store.productFilter
```

This means AngularJS will call `store.productFilter(product)` for each product. If the function returns `true`, the product is shown. If it returns `false`, the product is hidden.

10. API Calls With \$http

`$http` is AngularJS service for calling backend APIs.

```
$http.get('/api/products').then(function (response) {  
    vm.products = response.data;  
});
```

The meaning is:

1. Call backend endpoint `/api/products`.
2. Wait for response.
3. Store response data in `vm.products`.
4. AngularJS updates product cards automatically.

POST Example

```
$http.post('/api/auth/login', vm.loginForm)
```

This sends login form data to backend. The body contains email and password.

PATCH Example

```
$http.patch('/api/cart/' + item.id, {quantity: item.quantity})
```

This updates only the quantity of a cart item.

DELETE Example

```
$http.delete('/api/cart/' + item.id)
```

This removes one cart item.

11. Authentication Token in Frontend

After login, backend returns a token. The frontend saves it in browser storage:

```
localStorage.setItem('byteBazaarToken', vm.token);
```

Then it sends this token with future API calls:

```
$http.defaults.headers.common.Authorization = 'Bearer ' + vm.token;
```

This is why cart and orders APIs know which user is logged in.

12. HTML Structure Used in the Project

The HTML uses semantic tags where possible:

Tag	Meaning	Used For
<header>	Top section of page.	Project title and navigation.
<nav>	Navigation area.	Catalog, cart, orders, login/logout buttons.
<main>	Main content area.	All visible app screens.
<section>	Separate content block.	Catalog, account, cart, orders.
<article>	One independent item.	Product card, cart item, order card.
<form>	User input form.	Login, register, checkout.

13. Advanced HTML Concepts Used

Required Form Inputs

```
<input type="email" required>
```

required tells browser that input cannot be empty before form submit.

Email Input

```
<input type="email">
```

Browser checks that the entered value looks like an email.

Number Input With Minimum Value

```
<input type="number" min="1">
```

Used for cart quantity. It stops user from selecting values below 1 from browser controls.

Dynamic Image Source

```

```

ng-src is used instead of normal src because image URL comes from AngularJS data.

14. CSS Box Model

The CSS starts with:

```
* {  
  box-sizing: border-box;  
}
```

Normally, width does not include padding and border. With border-box, width includes content, padding, and border. This makes layouts easier to control.

15. CSS Flexbox Used

Flexbox is used for simple row alignment.

```
.topbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

Property	Meaning
display: flex	Places children in a row by default.
justify-content: space-between	Moves first item to left and last item to right.
align-items: center	Vertically centers items.
gap	Adds space between child elements.

16. CSS Grid Used

Grid is used for product cards, toolbar, login/register forms, and cart rows.

```
.product-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
  gap: 16px;  
}
```

This line is important:

```
repeat(auto-fit, minmax(250px, 1fr))
```

It means: create as many columns as possible. Each column should be minimum 250px and can grow to take remaining space.

Cart Row Grid

```
.line-item {  
  display: grid;  
  grid-template-columns: 70px 1fr 80px 110px auto;  
}
```

This creates columns for image, product details, quantity input, subtotal, and remove button.

17. Responsive Design

Responsive design means the page should work on mobile and desktop. This project uses a media query:

```
@media (max-width: 760px) {  
  .topbar,  
  .toolbar,  
  .auth-panel,  
  .line-item {  
    display: grid;  
    grid-template-columns: 1fr;  
  }  
}
```

When screen width is less than 760px, these layouts become single-column. This makes them easier to read on phone screens.

18. CSS Selectors Used

Selector	Meaning
button	Selects all button elements.
.topbar	Selects elements with class topbar.
button:hover	Selects button when mouse is over it.
button.primary	Selects buttons that also have class primary.
.product-card img	Selects images inside product cards only.

19. Complete Frontend Flow

Product Page Load

1. Browser opens index.html.
2. AngularJS library loads from CDN.
3. app.js creates Angular app and controller.
4. vm.loadProducts() calls backend.
5. Products are stored in vm.products.

6. `ng-repeat` displays product cards.

Search Product

1. User types in search input.
2. `ng-model="store.search"` updates JavaScript value.
3. AngularJS runs product filter again.
4. Only matching products remain visible.

Open Cart

1. User clicks Cart button.
2. `ng-click` calls `store.show('cart')`.
3. If user is not logged in, account screen opens.
4. If logged in, cart API is called.
5. Cart items are shown using `ng-repeat`.

20. Common Mistakes to Avoid

- Do not change `ng-app` name unless you also change it in `app.js`.
- Do not use `src="{{product.imageUrl}}"`; use `ng-src` for AngularJS images.
- Do not forget to include AngularJS before `app.js`.
- Do not call cart/order APIs without token.
- Do not remove `ng-model` from form inputs, or AngularJS will not receive typed values.

21. Study Order for This Frontend

1. Read `index.html` and find all `ng-` attributes.
2. Open `app.js` and find matching functions.
3. Open browser and test each button.
4. Study `$http.get`, `$http.post`, `$http.patch`, and `$http.delete`.
5. Then study CSS grid and flexbox layout in `styles.css`.

Best way to learn this project: keep `index.html` and `app.js` open side by side. For every button in HTML, find the function it calls in JavaScript.